PARTE 2: Encontrar y reparar enlaces incorrectos (35 Puntos)

2.1 Lista Simple

- **Estado defectuoso:** cabeza = A; siguiente: A → B; B → C; C → B
- **Dibujo del estado defectuoso:**

```
  cabeza → [ A | sig ] → [ B | sig ] → [ C | sig ] --+
                                  ^                  | (Ciclo
incorrecto)
                                  |__________________|
```

- **Referencia incorrecta:** C.siguiente → B (crea un ciclo infinito entre B y C).
- **Recorrido desde cabeza (primeras 5 visitas):** 1. A | 2. B | 3. C | 4. B | 5. C
- **Reparación y Dibujo Corregido:**

```
  cabeza → [ A | sig ] → [ B | sig ] → [ C | nulo ]  (Se asigna
C.siguiente = nulo)
```

- **Explicación:** Al reasignar C.siguiente = nulo, la lista recobra su comportamiento lineal terminado en C de forma válida.

2.2 Lista Circular Simple

- **Estado defectuoso:** cabeza = A; siguiente: A → B; B → nulo; C → A
- **Dibujo del estado defectuoso:**

```
  cabeza → [ A | sig ] → [ B | nulo ] (Corte)     [ C | sig ] --+
                ^
|

|_____________________________________________________________|
```

- **Referencia incorrecta:** B.siguiente → nulo (rompe el enlace circular antes de C).
- **Recorrido desde cabeza (primeras 5 visitas):** 1. A | 2. B *(Se detiene prematuramente en 2 visitas al encontrar nulo)*.
- **Reparación y Dibujo Corregido:**

```
  cabeza → [ A | sig ] → [ B | sig ] → [ C | sig ] --+  (Se asigna
B.siguiente = C)
                ^                                    |
                |____________________________________|
```

- **Explicación:** Al reconectar B.siguiente = C, se restituye la secuencia circular completa A → B → C → A.

2.3 Lista Doble Lineal

- **Estado defectuoso:** cabeza = A, cola = C
  siguiente: A → B; B → C; C → nulo
  anterior: A → nulo; B → C; C → B
- **Dibujo del estado defectuoso:**

```
             +----------------------+ (Puntero anterior
incorrecto)
             v                      |
  cabeza → [A] <=== sig/ant ===> [B] <=== sig/ant ===> [C] ← cola
             |                                          |
             +------------→ nulo <----------------------+
```

- **Referencia incorrecta:** B.anterior → C (apunta erróneamente hacia adelante a C en vez de hacia atrás a A).
- **Recorrido desde cola usando anterior (primeras 5 visitas):** 1. C | 2. B | 3. C | 4. B | 5. C
- **Reparación y Dibujo Corregido:**

```
  cabeza → [A] <=== sig/ant ===> [B] <=== sig/ant ===> [C] ← cola
             |                                          |
             +------------→ nulo <----------------------+
  (Se corrige B.anterior = A)
```

- **Explicación:** Al corregir B.anterior = A, el recorrido inverso desde la cola visita correctamente C → B → A → nulo.

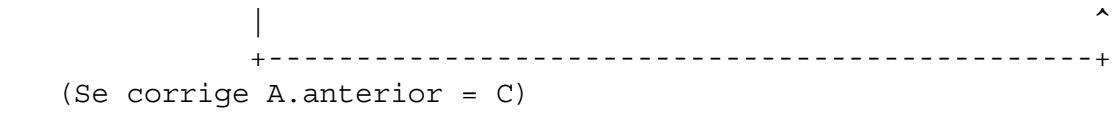
2.4 Lista Doble Circular

- **Estado defectuoso:** cabeza = A, cola = C
  siguiente: A → B; B → C; C → A
  anterior: A → B; B → A; C → B
- **Dibujo del estado defectuoso:**

```
  cabeza → [A] <=== sig/ant ===> [B] <=== sig/ant ===> [C] ← cola
             | ^                    ^                    |
             | +--------------------+ (Incorrecto)       |
             +-----------------------------------------+
```

- **Referencia incorrecta:** A.anterior → B (debería apuntar al último nodo, es decir, a cola = C).
- **Recorrido desde cabeza usando anterior (primeras 5 visitas):** 1. A | 2. B | 3. A | 4. B | 5. A
- **Reparación y Dibujo Corregido:**

```
             +-----------------------------------------+
             v                                         |
  cabeza → [A] <=== sig/ant ===> [B] <=== sig/ant ===> [C] ← cola
```



- **Explicación:** En una lista circular doble, el puntero anterior del primer nodo (cabeza A) debe referenciar al último nodo (cola C) para garantizar navegación bidireccional continua.